

FACULDADE DE INFORMÁTICA E ADMINISTRAÇÃO PAULISTA — FIAP

Pós-Graduação em Inteligência Artificial para Desenvolvedores

OTIMIZAÇÃO DE ROTAS MÉDICAS PARA DISTRIBUIÇÃO DE MEDICAMENTOS E INSUMOS

Algoritmos Genéticos aplicados ao TSP e VRP com restrições logísticas

Tech Challenge — Fase 2

Jefferson Antônio Pantoja Silva | jeffkd35@gmail.com

Wilson Lima da Silva | wilson.slima@gmail.com

Gustavo Lopes da Silva | gustavo_lsilva@hotmail.com

Felipe Soeiro Lopes | felipesoeiro.contato@outlook.com.br

Vinicius Tavares Sousa da Silva | viniciustavares2014@gmail.com

São Paulo

2026

RESUMO

O presente trabalho descreve o desenvolvimento de um sistema de otimização de rotas médicas para distribuição de medicamentos e insumos, utilizando Algoritmos Genéticos para resolver o Problema do Caixeiro Viajante (TSP) e o Problema de Roteamento de Veículos (VRP). O projeto parte de um código base de TSP e evolui em nove sprints incrementais para uma solução completa com restrições operacionais — prioridade de entrega, capacidade e autonomia dos veículos —, visualização interativa em tempo real com Pygame, camada de Large Language Models (LLM) para geração de relatórios e instruções em linguagem natural, e uma suite de experimentos comparativos. Cinco configurações de população foram avaliadas em modo VRP — pop50, pop100, pop100_no_elitism, pop500 e pop500_no_elitism — utilizando o dataset sintético de entregas. Todas as configurações atingiram o mesmo fitness final (0,16), com o cenário pop50 apresentando o melhor equilíbrio: convergência na geração 103 em apenas 1,323 segundos, contra 13,4 segundos do pop500. O sistema é validado por 62 testes automatizados e disponibiliza interface de linha de comando configurável para execução de experimentos reproduzíveis.

Palavras-chave: algoritmos genéticos; otimização de rotas; TSP; VRP; distribuição de medicamentos; Pygame; Large Language Models.

ABSTRACT

This paper describes the development of a medical route optimization system for the distribution of medicines and supplies, using Genetic Algorithms to solve the Travelling Salesman Problem (TSP) and the Vehicle Routing Problem (VRP). The project starts from a TSP code base and evolves over nine incremental sprints into a complete solution featuring operational constraints — delivery priority, vehicle capacity and vehicle autonomy —, real-time interactive visualization with Pygame, a Large Language Model (LLM) layer for natural-language report and instruction generation, and a comparative experiment suite. Five population configurations were evaluated in VRP mode — pop50, pop100, pop100_no_elitism, pop500 and pop500_no_elitism — using the synthetic deliveries dataset. All configurations reached the same final fitness (0.16), with pop50 delivering the best balance: convergence at generation 103 in only 1.323 seconds, compared to 13.4 seconds for pop500. The system is validated by 62 automated tests and provides a configurable command-line interface for reproducible experiment execution.

Keywords: genetic algorithms; route optimization; TSP; VRP; medicine distribution; Pygame; Large Language Models.

SUMÁRIO

1 INTRODUÇÃO

2 FUNDAMENTAÇÃO TEÓRICA

- 2.1 Algoritmos Genéticos
- 2.2 Problema do Caixeiro Viajante (TSP)
- 2.3 Vehicle Routing Problem (VRP)
- 2.4 Large Language Models

3 METODOLOGIA

4 IMPLEMENTAÇÃO

- 4.1 Estrutura do Projeto
- 4.2 Função de Fitness
- 4.3 Dados de Entrada
- 4.4 Execução
- 4.5 Modo VRP
- 4.6 Camada LLM

5 EXPERIMENTOS

6 RESULTADOS

7 TRABALHOS FUTUROS

REFERÊNCIAS

Repositório GitHub

Relatório de Resultados (HTML)



Repositório GitHub

<https://github.com/tech-challenge-fiap-GRUPO-2026/tech-challenge-fase2>



Relatório de Resultados (HTML)

[Página de resultados](#)

1 INTRODUÇÃO

A logística de distribuição de medicamentos e insumos em ambiente hospitalar e para unidades de saúde representa um desafio operacional relevante. A definição de rotas eficientes para veículos de entrega impacta diretamente o custo operacional, o cumprimento de prazos e a qualidade do atendimento ao paciente. Em contextos de urgência, a priorização incorreta de entregas pode resultar em falta de insumos críticos.

Problemas de roteamento de veículos são, em geral, NP-difíceis — não existe algoritmo polinomial que garanta a solução ótima para instâncias de grande porte. Nesse cenário, metaheurísticas como os Algoritmos Genéticos (AGs) se destacam como abordagens eficazes, capazes de encontrar soluções de alta qualidade em tempo computacionalmente viável.

Este trabalho apresenta o desenvolvimento do Projeto 2 do Tech Challenge Fase 2 da Pós-Graduação em Inteligência Artificial para Desenvolvedores da FIAP. O sistema implementa um resolvidor genético para TSP e VRP, com restrições operacionais de prioridade, capacidade e autonomia, integrado a uma camada LLM para geração de relatórios e instruções em linguagem natural, e validado por uma suite de experimentos comparativos.

As principais contribuições deste trabalho incluem: (i) arquitetura incremental em sprints, evoluindo de um resolvidor TSP simples até um sistema VRP completo com LLM; (ii) função de fitness multi-objetivo com quatro componentes de penalização logística; (iii) cromossomo de frota para evolução conjunta de distribuição e ordem no VRP; (iv) camada LLM testável, offline por padrão e com integração OpenAI opcional; e (v) análise experimental de cinco configurações VRP comparando convergência, tempo e qualidade da solução.

2 FUNDAMENTAÇÃO TEÓRICA

2.1 Algoritmos Genéticos

Algoritmos Genéticos são metaheurísticas de busca e otimização inspiradas no processo de seleção natural e na genética. Introduzidos por Holland (1975) e popularizados por Goldberg (1989), os AGs mantêm uma população de soluções candidatas (indivíduos) e a evoluem iterativamente por meio de operadores de seleção, crossover e mutação, avaliando cada indivíduo por uma função de aptidão (fitness).

No contexto deste projeto, cada indivíduo representa uma solução completa para o problema de roteamento. No modo TSP, o indivíduo é uma permutação das entregas. No modo VRP, o indivíduo representa uma frota completa, codificando simultaneamente a atribuição de entregas a veículos e a ordem de visita em cada rota.

2.2 Problema do Caixeiro Viajante (TSP)

O Problema do Caixeiro Viajante (Travelling Salesman Problem — TSP) busca encontrar a rota de menor custo que visita todos os pontos exatamente uma vez e retorna ao ponto de partida. É um dos problemas mais estudados em otimização combinatória, pertencente à classe NP-difícil (Garey e Johnson, 1979).

No contexto médico deste projeto, cada ponto representa um local de entrega de medicamentos ou insumos. O depósito representa o ponto de partida da rota, e o custo é composto pela distância euclidiana entre os pontos mais as penalidades operacionais.

2.3 Vehicle Routing Problem (VRP)

O Vehicle Routing Problem (VRP) generaliza o TSP para múltiplos veículos, cada um com sua própria rota. O objetivo é minimizar o custo total da frota, respeitando restrições de capacidade de cada veículo. O VRP é igualmente NP-difícil e surge naturalmente em problemas logísticos reais.

O módulo `src/routing/vrp.py` implementa o VRP com um cromossomo de frota: um único indivíduo representa toda a distribuição de entregas entre os veículos disponíveis. Os operadores de mutação incluem operações que movem entregas entre veículos, trocam entregas entre rotas e reordenam a sequência dentro de uma rota — permitindo explorar simultaneamente o espaço de distribuição e de ordenação.

2.4 Large Language Models

Large Language Models (LLMs) são modelos de linguagem de grande escala, treinados em vastos corpora textuais, capazes de gerar texto coerente e contextualmente relevante a partir de prompts. No contexto de logística e operações, LLMs podem ser utilizados para transformar dados estruturados em linguagem natural — relatórios, instruções e respostas a consultas.

A Sprint 7 implementou uma camada LLM testável em `src/llm/`, utilizando `references/agent-llm.py` como referência técnica. A camada suporta três tipos de saída: relatório operacional de frota, instruções por motorista e respostas a perguntas sobre as rotas. Por padrão, opera em modo offline com respostas determinísticas; com a injeção de um cliente compatível (OpenAI), realiza chamadas reais à API.

3 METODOLOGIA

O projeto foi conduzido em nove sprints incrementais, cada uma com objetivo técnico definido e artefatos verificáveis. A metodologia de desenvolvimento seguiu o padrão de evolução gradual, partindo de uma implementação mínima funcional (TSP) e adicionando funcionalidades progressivamente até a entrega final consolidada.

Sprint 1 — Análise do código base em references/ e elaboração do plano de extensão.

Sprint 2 — Migração do resolvidor TSP para src/ com dataclasses, type hints e testes automatizados com pytest.

Sprint 3 — Inclusão de prioridades de entrega (HIGH, MEDIUM, LOW) e penalização por atraso na função de fitness.

Sprint 4 — Restrição de capacidade máxima por veículo com penalização proporcional ao excesso.

Sprint 5 — Restrição de autonomia (distância máxima) por veículo com penalização por violação.

Sprint 6 — Implementação do modo VRP com cromossomo de frota e operadores de mutação inter-rota.

Sprint 7 — Camada LLM em src/llm/ para relatórios, instruções e Q&A — offline e com OpenAI opcional.

Sprint 8 — Runner de experimentos em src/metrics/: 5 configurações VRP com artefatos em artifacts/.

Sprint 9 — Consolidação: relatório técnico, roteiro de demonstração e manifesto de artefatos finais.

A validação foi conduzida com testes automatizados usando pytest em cada sprint. A última validação registrada apresentou 62 testes passando, cobrindo algoritmo genético, fitness, modelos de dados, leitura de CSV, visualização e camada LLM.

4 IMPLEMENTAÇÃO

4.1 Estrutura do Projeto

```
src/
  ga/genetic_algorithm.py  - algoritmo genético e operadores
  routing/tsp.py           - resolvidor TSP
  routing/vrp.py           - resolvidor VRP (cromossomo de frota)
  models/                 - Entrega, Veículo, Cidade, Prioridade
  visualization/          - Pygame (rota 2D) + Matplotlib (fitness)
  llm/                    - prompts, relatórios, instruções, Q&A
  metrics/                - runner de experimentos e exportação
  data_loader.py           - leitura de CSV
  main.py                  - CLI e demo visual
tests/                     - 62 testes automatizados
data/
  deliveries_sample.csv    - entregas sintéticas
  brazil_capitals_sample.csv - capitais brasileiras em plano 2D
  vehicles_sample.csv      - veículos com capacidade e autonomia
artifacts/
  experiments/             - resultados dos experimentos VRP
  final/                   - manifesto de artefatos
```

4.2 Função de Fitness

O fitness de cada solução é calculado como a soma da distância total percorrida pela rota (ou pela frota, no modo VRP) e das penalidades por violação de restrições operacionais. Soluções com menor fitness são consideradas melhores — o algoritmo minimiza esse valor.

```
fitness = distancia_total
         + penalidade_de_atraso
         + penalidade_de_capacidade
         + penalidade_de_autonomia
```

Tabela 1 — Componentes da Função de Fitness

Componente	Penalidade	Condição
Atraso HIGH	$100,0 \times \text{unidades}$	Entrega com prioridade alta em atraso
Atraso MEDIUM	$30,0 \times \text{unidades}$	Entrega com prioridade média em atraso
Atraso LOW	$10,0 \times \text{unidades}$	Entrega com prioridade baixa em atraso
Excesso de Capacidade	$25,0 \times \text{unidades}$	Carga acima da capacidade do veículo
Excesso de Autonomia	$25,0 \times \text{unidades}$	Distância acima do alcance do veículo

Fonte: elaborado pelos autores.

4.2.1 Função de Fitness no Modo VRP

No modo VRP, o fitness não é calculado para uma única rota, mas para toda a frota de veículos. Cada rota individual é avaliada com as restrições específicas do veículo que a executa (capacidade máxima e distância máxima independentes). A fórmula por rota é:

$$\begin{aligned}\text{fitness}(\text{rota_v}) = & \text{distância}(\text{rota_v}) \\ & + \text{penalidade_atraso}(\text{rota_v}) \\ & + \text{penalidade_capacidade}(\text{rota_v}, \text{veículo_v.max_capacity}) \\ & + \text{penalidade_autonomia}(\text{rota_v}, \text{veículo_v.max_distance})\end{aligned}$$

O fitness total da frota é a soma dos fitness individuais de todos os veículos:

$$\text{fitness_total} = \sum \text{fitness}(\text{rota_v}) \quad \text{para todo veículo } v \text{ na frota}$$

O algoritmo minimiza fitness_total — melhoras em qualquer rota individual contribuem diretamente para a melhora da solução de frota completa.

4.2.2 Modelo de Tempo de Chegada e Penalidade por Atraso

O cálculo do fitness percorre a sequência de entregas acumulando o tempo de chegada com base na distância euclidiana entre pontos consecutivos. A velocidade é normalizada: cada unidade de distância equivale a uma unidade de tempo. Para cada entrega visitada, o algoritmo verifica se o tempo de chegada excede o prazo (due_time) e aplica a penalidade correspondente à prioridade:

- Prioridade HIGH: $\text{penalidade} = (\text{chegada} - \text{due_time}) \times 100,0$
- Prioridade MEDIUM: $\text{penalidade} = (\text{chegada} - \text{due_time}) \times 30,0$
- Prioridade LOW: $\text{penalidade} = (\text{chegada} - \text{due_time}) \times 10,0$

A penalidade de atraso é aplicada somente quando $\text{chegada} > \text{due_time}$; caso contrário, o valor é zero. As constantes foram calibradas para que entregas HIGH atrasadas sejam 10 vezes mais penalizadas que entregas LOW, forçando o AG a priorizar rotas que respeitem os prazos críticos.

4.2.3 Penalidades de Capacidade e Autonomia

Além das penalidades por atraso, dois fatores operacionais são penalizados proporcionalmente ao excesso em relação ao limite do veículo:

- Excesso de capacidade: se o peso total das entregas da rota exceder max_capacity do veículo, aplica-se $(\text{peso_total} - \text{max_capacity}) \times 25,0$. Constante: CAPACITY_EXCESS_PENALTY = 25,0.
- Excesso de autonomia: se a distância total da rota fechada (incluindo retorno ao depósito) exceder max_distance do veículo, aplica-se $(\text{distância} - \text{max_distance}) \times 25,0$. Constante: DISTANCE_EXCESS_PENALTY = 25,0.

As penalidades são aditivas: uma rota pode acumular simultaneamente penalidade de atraso, de capacidade excedida e de autonomia excedida. Isso garante que o AG busque soluções que respeitem múltiplas restrições ao mesmo tempo, sem que a violação de uma restrição mascare outra.

4.2.4 Distância Euclidiana e Rota Fechada

A distância entre dois pontos é calculada como a distância euclidiana 2D: $\text{dist}(p_1, p_2) = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$. A rota é considerada fechada — o último ponto conecta de volta ao primeiro — garantindo o retorno ao depósito. No modo VRP, quando um depósito é especificado, ele é inserido no início de cada rota antes do cálculo do fitness: [depósito, entrega₁, entrega₂, ..., entrega_N], com retorno ao depósito no fechamento.

4.3 Dados de Entrada

O projeto utiliza três datasets de exemplo em formato CSV, localizados no diretório data/:

- **deliveries_sample.csv**: entregas sintéticas com coordenadas, peso e prazo
- **brazil_capitals_sample.csv**: capitais brasileiras em plano cartesiano 2D para simulação visual com fundo do Brasil
- **vehicles_sample.csv**: veículos com capacidade máxima de carga e autonomia máxima em distância

4.4 Execução

Exemplo de execução no modo TSP com capitais brasileiras:

```
.venv/bin/python -m src.main \  
  --deliveries-file data/brazil_capitals_sample.csv \  
  --vehicle-id 3 --population-size 200 \  
  --mutation-probability 0.3 --elite-size 2 --fps 15
```

Exemplo no modo VRP com múltiplos veículos:

```
.venv/bin/python -m src.main \  
  --mode vrp \  
  --deliveries-file data/brazil_capitals_sample.csv \  
  --vehicle-ids 1 3 5 --population-size 100 \  
  --mutation-probability 0.3 --elite-size 2 --fps 15
```

4.5 Modo VRP

O modo VRP implementa um cromossomo de frota: cada indivíduo na população representa uma solução completa para toda a frota, codificando qual veículo realiza cada entrega e em que ordem. O processo evolutivo é:

1. criar uma população inicial de frotas completas com distribuição aleatória;
2. avaliar cada frota pelo fitness agregado (soma das rotas individuais);
3. aplicar crossover trocando sub-rotas entre soluções de frota;
4. aplicar mutações: mover entrega entre veículos, trocar entregas e reordenar sequência;
5. manter as melhores frotas por elitismo para a próxima geração.

4.5.1 Crossover de Frota — `fleet_crossover`

O crossover de frota (implementado em `src/routing/vrp.py` — função `fleet_crossover`) combina dois cromossomos de frota (pai 1 e pai 2) em três etapas:

1. Achatamento (`flatten`): cada cromossomo de frota — que é uma lista de listas — é convertido em uma sequência global única de entregas. Por exemplo, se o pai 1 tem veículo A com [E1, E2] e veículo B com [E3, E4, E5], a sequência achatada é [E1, E2, E3, E4, E5].
2. Order Crossover (OX) sobre as sequências globais: aplica-se o operador OX clássico sobre as duas sequências achatadas. O OX escolhe dois pontos de corte aleatórios [`start`, `end`] e copia o segmento do pai 1 nesse intervalo para o filho. As posições restantes são preenchidas com os genes do pai 2 na ordem em que aparecem, pulando os genes já presentes no filho. Isso garante que cada entrega apareça exatamente uma vez no filho — sem duplicatas ou omissões.
3. Divisão por tamanhos herdados: a sequência filha (ainda achatada) é repartida em sub-rotas usando os tamanhos de um dos pais, escolhido aleatoriamente com probabilidade 50%. Os tamanhos representam quantas entregas cada veículo recebe. Se a soma dos tamanhos for menor que o total de entregas (caso de arredondamento), as entregas excedentes são alocadas à última rota.

Exemplo com 3 veículos e 6 entregas:

Pai 1: [V1: E1, E2] [V2: E3, E4] [V3: E5, E6] → `tamanhos` = [2, 2, 2]

Pai 2: [V1: E4, E1, E6] [V2: E2, E5] [V3: E3]

Sequências achatadas:

Pai 1: [E1, E2, E3, E4, E5, E6]

Pai 2: [E4, E1, E6, E2, E5, E3]

OX (corte em [2, 5]):

Segmento copiado do Pai 1: [E3, E4, E5]

Restantes do Pai 2 (excluindo E3, E4, E5): [E1, E6, E2]

Filho achatado: [E1, E6, E3, E4, E5, E2]

Divisão com tamanhos do Pai 1 [2, 2, 2]:

Filho: [V1: E1, E6] [V2: E3, E4] [V3: E5, E2]

Esta estratégia combina o que há de melhor nos dois pais: a ordem relativa das entregas é herdada via OX (preservando sub-rotas eficientes do pai), enquanto a distribuição entre veículos é herdada de um dos pais (preservando o balanceamento de carga que já foi selecionado pela pressão evolutiva).

4.5.2 Mutação de Frota — `mutate_fleet`

A mutação de frota (função `mutate_fleet` em `src/routing/vrp.py`) é aplicada com probabilidade `mutation_probability` a cada cromossomo filho gerado. Quando ativada, escolhe aleatoriamente entre três tipos de mutação, cada um projetado para explorar um aspecto diferente do espaço de busca do VRP:

Tipo 1 — Mover entrega (`move`):

Retira uma entrega aleatória de uma rota não-vazia e a insere em uma posição aleatória de qualquer outra rota (inclusive a mesma). Esta mutação altera a distribuição de carga entre veículos, podendo corrigir desequilíbrios de capacidade ou aproximar entregas geograficamente próximas ao mesmo veículo.

Implementação: `origin = random.choice(non_empty_routes); delivery = mutated[origin].pop(random position); mutated[destination].insert(random position, delivery).`

Tipo 2 — Trocar entregas (`swap`):

Seleciona duas rotas não-vazias distintas e troca uma entrega aleatória de cada uma. Ao contrário do "move", esta operação preserva o número de entregas por veículo, alterando apenas quais entregas cada veículo realiza. É útil para explorar diferentes atribuições de entregas sem desbalancear a carga.

Implementação: `route_a, route_b = random.sample(non_empty_routes, 2); mutated[route_a][i], mutated[route_b][j] = mutated[route_b][j], mutated[route_a][i].`

Tipo 3 — Reordenar rota (`reorder`):

Seleciona uma rota não-vazia e troca duas entregas adjacentes (`swap` de posições consecutivas). Esta operação não altera a distribuição entre veículos — apenas a sequência de visita dentro de uma rota. É análoga à mutação por `swap` do TSP e explora o espaço de ordenação local.

Implementação: `index = random.randrange(len(route) - 1); route[index], route[index + 1] = route[index + 1], route[index].`

Os três tipos garantem que o AG explore simultaneamente: (a) redistribuição de entregas entre veículos, (b) trocas de atribuição e (c) reordenação dentro das rotas. A seleção aleatória uniforme entre os tipos evita que o algoritmo fique preso em um único modo de exploração. A mutação é aplicada apenas uma vez por cromossomo por geração — não de forma acumulativa.

4.5.3 População Inicial do VRP

A população inicial do VRP combina duas estratégias para equilibrar qualidade inicial e diversidade genética:

- 1 solução heurística determinística: as entregas são ordenadas por prioridade (HIGH → MEDIUM → LOW), depois por prazo (due_time crescente) e por fim por id. Essa sequência é distribuída pelos veículos tentando respeitar max_capacity: a entrega vai para o veículo com menor carga atual que ainda cabe a entrega; se nenhum couber, vai para o veículo com menor carga total. Esta solução garante que a população inicie com pelo menos uma configuração sensata.
- N-1 soluções aleatórias: cada entrega é alocada a um veículo escolhido aleatoriamente, sem respeitar capacidade ou prioridade. A ordem dentro de cada rota também é aleatória. Isso garante diversidade genética suficiente para que o AG explore o espaço de busca sem convergir prematuramente.

4.5.4 Elitismo no VRP

O elitismo preserva os elite_size melhores cromossomos de frota (menor fitness_total) diretamente para a próxima geração, sem crossover nem mutação. Os demais (population_size – elite_size) filhos são gerados via fleet_crossover + mutate_fleet, com pais selecionados aleatoriamente do parent_pool (os parent_pool_size melhores indivíduos da geração atual). Este mecanismo garante que o melhor resultado encontrado nunca se perde entre gerações.

A visualização do modo VRP anima a evolução geracional, desenha uma rota por veículo com cores distintas, revela o traçado progressivamente e exibe o histórico agregado de fitness da frota. Com o dataset de capitais brasileiras, a tela exibe um fundo simplificado do mapa do Brasil.

4.6 Camada LLM

A camada LLM em src/llm/ monta contexto textual a partir de TSPSolution ou VRPSolution e suporta três tipos de saída: relatório operacional, instruções por motorista e respostas a perguntas sobre rotas. Sem cliente externo, retorna respostas determinísticas. Com cliente injetado ou --provider openai, envia mensagens ao provedor LLM configurado.

```
# Relatório VRP offline
.venv/bin/python -m src.llm \
  --mode vrp --output report \
  --deliveries-file data/brazil_capitals_sample.csv

# Com OpenAI
.venv/bin/python -m src.llm \
  --provider openai --model gpt-4o-mini \
  --mode vrp --output report \
  --deliveries-file data/brazil_capitals_sample.csv
```

5 EXPERIMENTOS

A Sprint 8 foi concluída com um runner em `src/metrics/` que executa e compara cinco configurações de população em modo VRP no dataset de entregas sintéticas. As configurações variam em tamanho de população, probabilidade de mutação e uso de elitismo.

Tabela 2 — Configurações dos Experimentos VRP

Configuração	Pop.	Mutação	Crossover	Elitismo	Pool	Gerações
pop50	50	0,14	0,68	1	6	500
pop100	100	0,08	0,80	2	10	500
pop100 no elitism	100	0,08	0,80	0	10	500
pop500	500	0,02	0,90	6	20	500
pop500 no elitism	500	0,02	0,90	0	20	500

Fonte: elaborado pelos autores.

Os resultados comparativos são apresentados na Tabela 3:

Tabela 3 — Resultados Comparativos dos Experimentos VRP

Configuração	Fitness Final	Convergência	Tempo	Melhoria
pop50 ★	0,16	103	1,323s	0,40
pop100	0,16	187	2,588s	0,40
pop100 no elitism	0,16	176	2,639s	0,40
pop500	0,16	33	13,404s	0,35
pop500 no elitism	0,16	33	13,425s	0,35

Fonte: elaborado pelos autores. ★ = melhor equilíbrio tempo/convergência.

5.1 Pergunta de Pesquisa

Como o tamanho da população e a presença de elitismo afetam a qualidade da solução, a velocidade de convergência e o custo computacional do AG no problema VRP com restrições operacionais (prioridade, capacidade e autonomia)?

5.2 Setup Experimental

Todas as configurações foram executadas sobre o mesmo dataset (`data/deliveries_sample.csv`, com todos os veículos de `vehicles_sample.csv`), com semente fixa para reprodutibilidade, durante 500 gerações cada. As métricas coletadas foram: fitness final do melhor indivíduo, geração de convergência (última geração com melhoria do best fitness), tempo total de execução em segundos e melhoria relativa do fitness $(\text{fitness_inicial} - \text{fitness_final}) / \text{fitness_inicial}$.

5.3 Análise: Robustez do Algoritmo

Todas as 5 configurações atingiram fitness final = 0,16, com desvio padrão zero. Este resultado demonstra que o AG implementado é robusto: independentemente do tamanho da população (50 a 500 indivíduos), a solução de qualidade equivalente é encontrada. Isso ocorre porque o espaço de busca do dataset utilizado é suficientemente pequeno para que todas as configurações converjam para a mesma bacia de atração do ótimo local.

5.4 Análise: Custo por Geração

O custo computacional por geração escala aproximadamente de forma linear com o tamanho da população, conforme esperado para um AG com avaliação de fitness $O(N)$:

- pop50: 1,323s / 500 gerações $\approx$ 2,6 ms/geração
- pop100: 2,588s / 500 gerações $\approx$ 5,2 ms/geração ($\approx 2\times$ pop50)
- pop500: 13,404s / 500 gerações $\approx$ 26,8 ms/geração ($\approx 10\times$ pop50)

A linearidade confirma que o gargalo é a avaliação dos fitness individuais — o crossover e a mutação têm custo desprezível em comparação. Isso implica que, para datasets maiores, otimizações na função de fitness (caching, vetorização com NumPy) teriam impacto direto na escalabilidade do sistema.

5.5 Análise: pop50 como Melhor Custo-Benefício

A configuração pop50 entregou o mesmo fitness que pop500 em 10 vezes menos tempo:

- pop50: 1,323s $\rightarrow$ fitness 0,16 $\rightarrow$ convergência na geração 103
- pop500: 13,404s $\rightarrow$ fitness 0,16 $\rightarrow$ convergência na geração 33

Economia de tempo: $13,404 - 1,323 = 12,081$ s (redução de 91%)

Speedup: $13,404 / 1,323 \approx 10,1\times$

Embora pop500 convirja em apenas 33 gerações (versus 103 do pop50), o custo total é muito maior porque cada geração com 500 indivíduos é $\sim 10\times$ mais cara. Para aplicações com necessidade de execução rápida — como demonstrações ao vivo ou sistemas em produção com restrições de tempo de resposta — pop50 é a configuração recomendada.

5.6 Análise: Impacto do Elitismo

Os pares com e sem elitismo revelam um comportamento inesperado: a ausência de elitismo não prejudicou a qualidade final e, no caso do par pop100, acelerou levemente a convergência:

- pop100 (com elitismo, elite_size=2): convergência na geração 187, tempo 2,588s
- pop100_no_elitism (sem elitismo): convergência na geração 176, tempo 2,639s
 $\rightarrow$ sem elitismo convergiu 11 gerações antes (+5,9% mais rápido em gerações)
- pop500 (com elitismo, elite_size=6): convergência na geração 33, tempo 13,404s
- pop500_no_elitism (sem elitismo): convergência na geração 33, tempo 13,425s
 $\rightarrow$ comportamento idêntico

A interpretação é que, no dataset utilizado, a pressão seletiva exercida pela seleção de pais (parent_pool com os melhores indivíduos) já é suficiente para guiar a convergência sem necessidade de preservação explícita da elite. Em datasets mais complexos ou com maior ruído na função de fitness, o elitismo tende a ser mais relevante para evitar a perda de soluções promissoras.

5.7 Artefatos dos Experimentos

O runner em src/metrics/ gera automaticamente os seguintes artefatos reproduzíveis:

- artifacts/experiments/sprint8_summary.csv — tabela comparativa em formato CSV
- artifacts/experiments/sprint8_summary.json — dados estruturados para integração
- artifacts/experiments/sprint8_summary.md — resumo legível em Markdown
- artifacts/charts/fitness_curves.png — curvas de convergência por configuração
- artifacts/charts/final_fitness.png — comparativo do fitness final
- artifacts/charts/execution_time.png — comparativo do tempo de execução

Comando para reproduzir os experimentos:

```
.venv/bin/python -m src.metrics \  
--deliveries-file data/deliveries_sample.csv \  
--vehicles-file data/vehicles_sample.csv \  
--output-dir artifacts
```

NOTA: Os artefatos gerados pelos experimentos estão em artifacts/experiments/ (CSV, JSON e Markdown) e em artifacts/charts/ (gráficos de curvas de convergência, fitness final e tempo de execução).

6 RESULTADOS

Os resultados obtidos demonstram a viabilidade do Algoritmo Genético como abordagem de otimização para o problema de distribuição de medicamentos. O sistema entregue cobre completamente os requisitos do enunciado da Fase 2:

- TSP funcional com visualização interativa em tempo real (Pygame);
- VRP com múltiplas rotas, cromossomo de frota e evolução conjunta;
- fitness multi-objetivo com distância, prioridade, capacidade e autonomia;
- leitura de entregas e veículos via CSV configurável;
- camada LLM testável para relatórios, instruções e Q&A;
- suite de 5 experimentos VRP com artefatos completos;
- visualização com fundo do mapa do Brasil (dataset de capitais);
- CLI configurável com múltiplos parâmetros;
- 62 testes automatizados passando.

Quanto aos experimentos VRP, todas as cinco configurações atingiram o mesmo fitness final (0,16), confirmando a robustez do algoritmo. O cenário pop50 se destacou pelo melhor equilíbrio entre qualidade e eficiência computacional, convergindo 10× mais rápido que pop500 sem perda de qualidade na solução. O pop500, apesar de convergir em apenas 33 gerações, demandou 13,4 segundos — mais de 10× o tempo do pop50 —, evidenciando a escalabilidade como fator crítico na escolha da configuração para ambientes de produção.

6.1 Métricas de Qualidade Consolidadas

A tabela a seguir consolida as métricas de qualidade do projeto completo:

- Testes automatizados: 62 passed (100% de aprovação, zero falhas)
- Fitness final VRP (todas as 5 configurações): 0,16
- Desvio padrão do fitness entre configurações: 0,00
- Melhor tempo de execução (pop50): 1,323s
- Speedup pop50 vs pop500: $\approx 10\times$
- Melhoria de fitness (pop50 e pop100): 0,40 (40%)
- Melhoria de fitness (pop500): 0,35 (35%)
- Sprints concluídas: 9 de 9
- Modos LLM implementados: 3 (report, instructions, question)
- Configurações de experimento testadas: 5

6.2 Validação por Testes Automatizados

A suite de testes cobre todos os componentes críticos do sistema:

- Algoritmo Genético (~15 testes): crossover OX, mutação por swap, população inicial aleatória, histórico de fitness, preservação de elite.
- Função de Fitness (~10 testes): distância euclidiana, penalidade por atraso HIGH/MEDIUM/LOW, penalidade de capacidade, penalidade de autonomia, rota fechada.
- Leitura de Dados (~8 testes): CSV de entregas, CSV de veículos, campos opcionais, validação de tipos.
- CLI (~6 testes): parsing de argumentos, modo TSP/VRP, seleção de veículos por --vehicle-ids.
- VRP (~12 testes): distribuição de entregas, fitness agregado, mutação inter-rota (move/swap/reorder), população de frota, iterate_vrp.
- Camada LLM (~11 testes): construção de prompts, respostas offline, fallback sem API key, modos TSP e VRP.

6.3 Conclusão dos Experimentos VRP

Os experimentos confirmam três conclusões principais:

1. Robustez: o AG encontra soluções de qualidade equivalente (fitness 0,16) independentemente do tamanho da população, desde que o número de gerações seja suficiente. Isso valida a implementação dos operadores genéticos VRP (crossover de frota, mutação inter-rota) como corretos e eficazes.
2. Eficiência: para o dataset utilizado, populações menores (pop50) são significativamente mais eficientes que populações grandes (pop500), com speedup de $\sim 10\times$ sem perda de qualidade. Recomenda-se começar com populações menores e aumentar somente se a qualidade for insuficiente.
3. Elitismo: não foi fator determinante neste dataset — a pressão seletiva do parent_pool já é suficiente. Em problemas com paisagens de fitness mais ruidosas (muitos ótimos locais), o elitismo tende a ser mais relevante para estabilização.

7 TRABALHOS FUTUROS

1. Refinamento de operadores genéticos do VRP para preservar agrupamentos geográficos (operadores de cluster-first route-second).
2. Integração com dados reais de malha viária — uso de OSMnx ou APIs de roteirização para distâncias reais em vez de distâncias euclidianas.
3. Implementação de cliente OpenAI concreto para demonstrações com chamadas reais à API de LLM.
4. Avaliação de algoritmos híbridos (AG + busca local, AG + heurísticas construtivas) para melhoria da convergência.
5. Expansão para VRPTW (VRP com janelas de tempo) para modelagem mais precisa de restrições de prazo.

REFERÊNCIAS

- GAREY, M. R.; JOHNSON, D. S.** Computers and Intractability: A Guide to the Theory of NP-Completeness. New York: W. H. Freeman, 1979.
- GOLDBERG, D. E.** Genetic Algorithms in Search, Optimization and Machine Learning. Reading, MA: Addison-Wesley, 1989.
- HOLLAND, J. H.** Adaptation in Natural and Artificial Systems. Ann Arbor: University of Michigan Press, 1975.
- LAPORTE, G.** The vehicle routing problem: An overview of exact and approximate algorithms. European Journal of Operational Research, v. 59, n. 3, p. 345–358, 1992.
- TOTH, P.; VIGO, D. (Ed.).** The Vehicle Routing Problem. Philadelphia: SIAM, 2002.
- OPENAI.** GPT-4 Technical Report. 2023. Disponível em: <<https://openai.com>>. Acesso em: jun. 2026.
- PYGAME COMMUNITY.** Pygame Documentation. 2023. Disponível em: <<https://www.pygame.org/docs/>>. Acesso em: jun. 2026.
- SCIKIT-LEARN.** User Guide — scikit-learn 1.2. 2023. Disponível em: <<https://scikit-learn.org>>. Acesso em: jun. 2026.